

SMA + Stigmergy 3D Drone Swarm — Debugging & Enhancement Session

Date: July 2, 2026 **Project:** swarm_project — 2-drone SMA + Stigmergy reactive obstacle avoidance (PX4 SITL + Gazebo Harmonic, WSL2)

Summary

This session covered three areas of work:

1. Diagnosed and fixed a critical bug causing drones to permanently freeze after their first obstacle encounter, silently corrupting every run from that point onward.
 2. Added visual origin/target markers to the simulation world for debugging and demo/paper figures.
 3. Identified a follow-on issue (fitness value blow-up near the obstacle threshold boundary) surfaced by the freeze fix, currently under investigation.
-

1. Critical Bug: Drones Freeze Permanently After First Obstacle Contact

Symptom

Across multiple independent runs (100 iterations each), the exact same pattern appeared every time a drone entered obstacle range:

- `LiDAR[i].min_range` locked to one exact value and never changed again for the rest of the run (sometimes 80–95 consecutive iterations).
- `x_best` stopped updating entirely once any drone triggered obstacle avoidance.
- Danger pheromone deposit position repeated at the exact same coordinate, iteration after iteration.
- `DangerPheromone.total_deposits` kept incrementing even though nothing else in the system was changing — a strong signal that *part* of the reactive loop was running while the drone's actual physical state was frozen.

This was confirmed reproducible across three separate runs with different obstacle geometries and random seeds — ruling out algorithm stochasticity (SMA's exploration/exploitation convergence) as the cause.

Root Cause

In `algorithm/sma_core.py`, the reactive avoidance layer correctly computed a repulsion vector and moved the drone away from the obstacle:

```
if rep_norm > 1e-8:
```

```
    new_positions[i] = positions[i] + repulsion
```

```
    ...
```

```
    continue # skip SMA update — reactive takes priority
```

However, at the end of `sma_step()`, **every** drone's new position — including ones that took the reactive/safety branch — was passed through the same greedy fitness-based acceptance check used for exploratory SMA moves:

```
new_fitness = fitness(new_positions, lidar_subscribers, danger_field)
```

```
accept = new_fitness < fitness_vals
```

```
final_pos = np.where(accept[:, None], new_positions, positions)
```

A repulsion move pushes a drone *away* from the target (since it moves away from an obstacle, not toward the goal), so `distance_to_target` in the fitness function increases. Compounding this, `fitness()` was evaluated on the new position **before** the LiDAR sensor was updated with that new position — so the obstacle-distance component of the fitness score still reflected the *old*, close-range reading, and didn't credit the escape move for improved safety.

Net effect: `new_fitness > fitness_vals[i]` almost every time → the move was rejected → the drone's position silently reverted to where it started. Next iteration: same position → same LiDAR reading → same repulsion vector computed → same rejection. This repeated indefinitely, producing exactly the frozen-telemetry symptom seen in the logs, while the pheromone deposit counter kept incrementing because that logic runs unconditionally, before the (broken) greedy check.

Fix

Reactive/repulsion moves are safety overrides, not candidate solutions to be scored against the objective function. The fix tracks which drones took the reactive branch in a given iteration and force-accepts their moves regardless of the fitness comparison:

```
reactive_mask = np.zeros(n, dtype=bool) # new: tracks safety-override moves
```

```
for i in range(n):
```

```
    if ...obstacle detected...:
```

```
        new_positions[i] = positions[i] + repulsion
```

```
        reactive_mask[i] = True      # new: mark as safety move
```

```
        ...
```

```
        continue
```

```
...
```

```
accept = new_fitness < fitness_vals
```

```
accept = accept | reactive_mask      # new: safety moves always accepted
```

```
final_pos = np.where(accept[:, None], new_positions, positions)
```

```
final_fit = np.where(accept, new_fitness, fitness_vals)
```

File changed: algorithm/sma_core.py (3 additions, no restructuring) **Files reviewed, unchanged (no bug found):** sensors/lidar.py, main.py

Verification

After deploying the fix and re-running:

- LiDAR ranges now change iteration-to-iteration instead of freezing on contact (e.g. Drone 0 progressed 7.16m → 7.34m → 7.48m → ... → 8.00m over ~25 iterations of a real escape trajectory).
- `x_best` continued updating *after* drones had passed through obstacle encounters (confirmed at iterations 65, 70, and 98 in the post-fix run) — this never occurred in any pre-fix log.

2. World Markers: Visual Origin/Target Indicators

Added flat, non-colliding "X" markers to windy.sdf to visually indicate the swarm's spawn point and target position in Gazebo, for debugging and paper/demo figures.

Design decisions

- **Flat, painted-on-ground geometry** (two crossed thin boxes at $\pm 45^\circ$) rather than 3D shapes (initial cylinder/sphere version was visually intrusive and looked like an obstacle).
- **No <collision> tag** — purely visual <visual> elements so the markers never interfere with the GPS-proximity obstacle sensor or PX4/Gazebo physics.
- **Color-coded:** green X at the origin [0, 0], gold X at the target [40, 40], both with emissive materials so they remain visible without relying on scene lighting.
- Inserted as <model> elements directly into windy.sdf, at the same nesting level as the five existing obstacle_N blocks — no PX4 rebuild required, changes take effect on next gz relaunch.

File changed: Tools/simulation/gz/worlds/windy.sdf (PX4-Autopilot, not part of swarm_project repo)

3. Open Issue: Fitness Value Blow-Up Near Obstacle Threshold (Under Investigation)

The freeze fix surfaced a secondary, previously-masked issue: in the most recent run, Best fitness spiked from 48.4 to 808+ over a handful of iterations while a drone hovered right at the exact LIDAR_DANGER_THRESHOLD boundary (min_range oscillating around 8.00m), then dropped back to sane values (5.9, 26.3) once the drone moved clear.

Suspected cause: the obstacle-penalty term inside fitness() is likely being evaluated against a stale LiDAR reading (the same staleness issue implicated in the original freeze bug — fitness() is called before lidars[i].update_position() runs for the new position), possibly combined with a penalty term that scales poorly (e.g. inverse-distance) right at the threshold edge.

Also noted: the final printed Final best fitness value did not match fitness(x_best) in the latest run, because it reads fitness_vals.min() from a population state that includes contaminated candidates. Recommend logging fitness(x_best.reshape(1,3), lidars, danger_field) explicitly at run-end for reporting purposes rather than trusting fitness_vals.min().

Status: Pending review of algorithm/fitness.py to confirm the exact source of the spike and apply a fix.

Files Changed This Session

File	Status	Change
algorithm/sma_core.py	Modified	Reactive-move greedy-selection bypass (freeze fix)
Tools/simulation/gz/worlds/windy.sdf	Modified	Added origin/target X markers
sensors/lidar.py	Reviewed	No changes — confirmed not the source of the freeze
main.py	Reviewed	No changes — confirmed not the source of the freeze
algorithm/fitness.py	Pending review	Suspected source of threshold-boundary fitness spike

Next Steps

1. Review algorithm/fitness.py to resolve the threshold-boundary fitness spike.
2. Consider adding hysteresis to LIDAR_DANGER_THRESHOLD (e.g. separate enter/exit thresholds) to reduce oscillation once a drone is near an obstacle edge — a minor tuning item, lower priority than the fitness spike.
3. Re-run the 3-condition experiment (baseline / obstacles-only / obstacles+wind) referenced in the original project plan once the fitness function is confirmed stable.